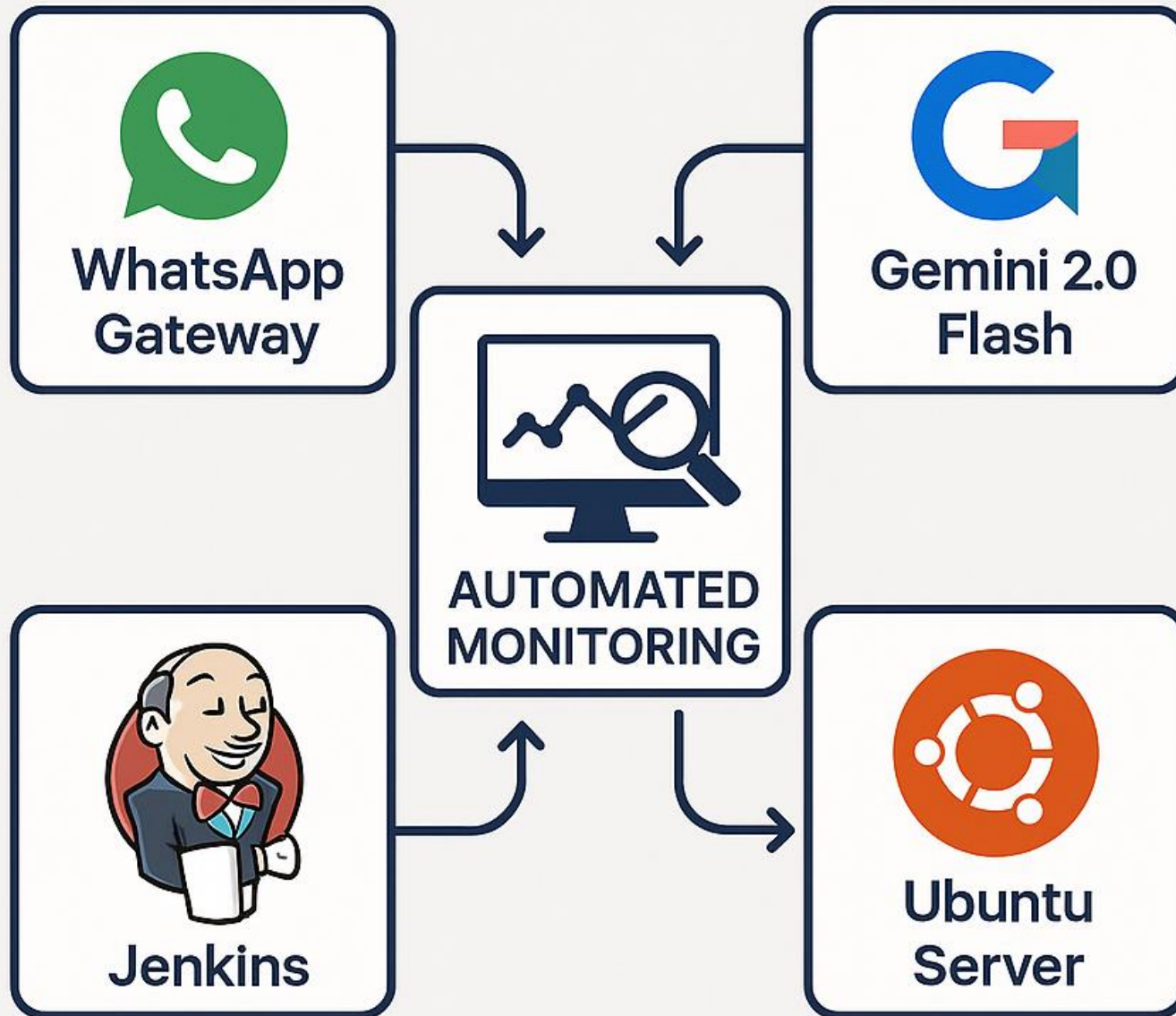


STEP-BY-STEP GUIDE TO BUILDING AN AUTOMATED BRUTE-FORCE SSH MONITORING SYSTEM

AUTOMATED MONITORING



Conceptual Diagram

This diagram shows how these tools were integrated for automated monitoring. At the center is the monitoring system, which receives inputs from Ubuntu Server (for system events) and Jenkins (for automation).

Gemini 2.0 Flash provides AI-powered analysis, while WhatsApp Gateway sends alerts to users.

Together, these techs form a real-time, intelligent and responsive monitoring ecosystem.

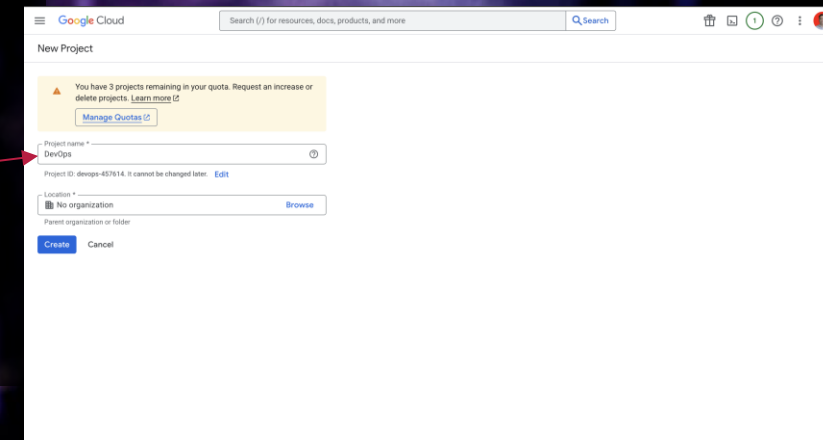
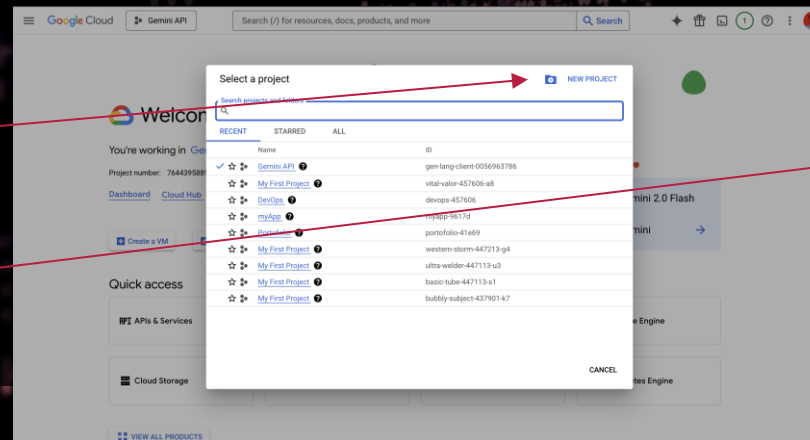
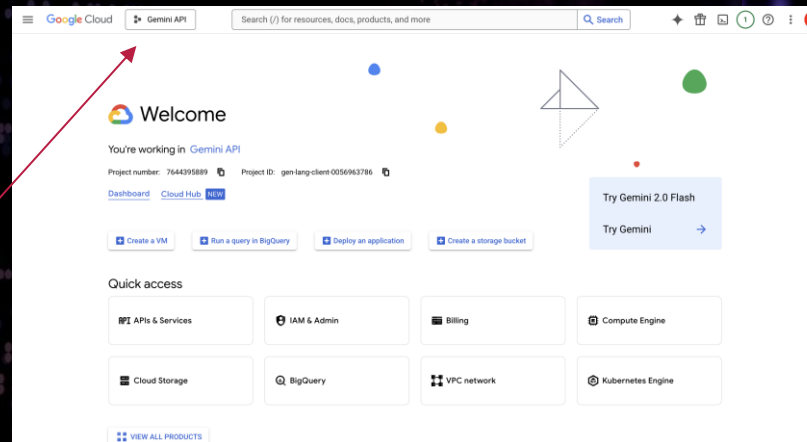
AGENDA OVERVIEW

- Obtaining and Configuring Gemini 2.0 Flash API
- Setting Up Fonnte API for WhatsApp Notifications
- Membuat Script Python (monitor.py)
- Uploading monitor.py to GitHub
- Configuring Jenkins for Automation

OBTAINING AND CONFIGURING GEMINI 2.0 FLASH API

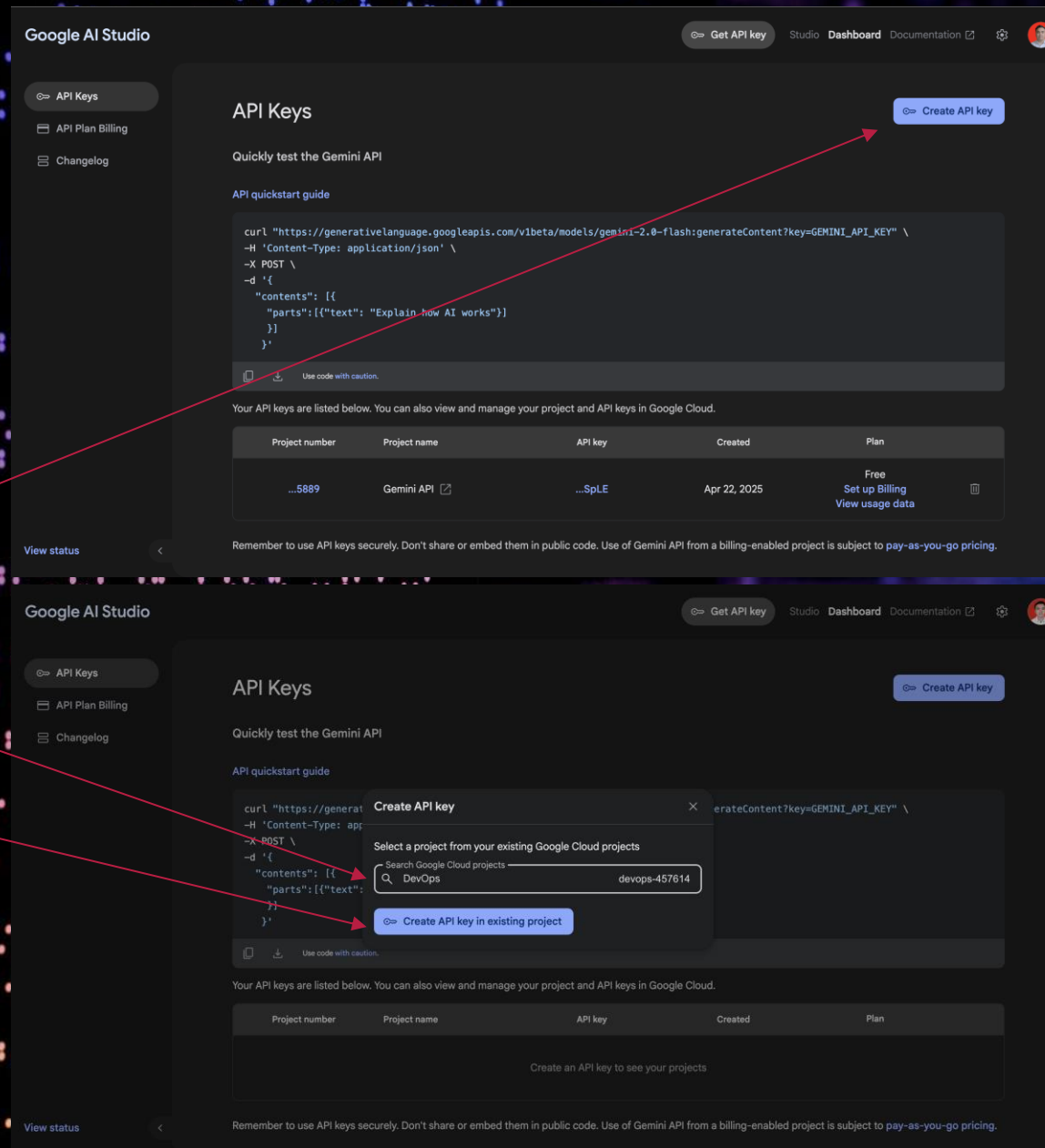
AKTIFKAN GOOGLE CLOUD & API GEMINI

1. Buka <https://console.cloud.google.com>,
Jangan lupa untuk login
2. Buat Project Baru (misal: DevOps):
 1. Klik pada bagian sebelah tulisan
GoogleCloud
 2. Lalu Klik New Project
 3. Lalu isi project namanya (misal:
DevOps). Lalu tekan tombol
create.



AMBIL API KEY

1. Masuk ke <https://makersuite.google.com/app/apikey>
2. Klik "Create API Key":
 1. Lalu pilih project DevOps yang telah kita buat sebelumnya
 2. Lalu klik Create API key in Existing Project
3. Salin dan simpan baik-baik



SETTING UP FONNTE API FOR WHATSAPP NOTIFICATIONS

CARA DAPATKAN FONNTE TOKEN API

1. **Kunjungi Website Fonnte.** <https://fonnte.com>
2. **Login / Daftar Akun**
 - a. Kalau belum punya, daftar dulu.
 - b. Setelah login, kamu bakal masuk ke **Dashboard**.
3. **Masuk ke Halaman Token API**
 - a. Di Dashboard → Klik menu **"Device"**
 - b. Tekan tombol Add Device, lalu masukkan nama device dan nomor device
 - c. Lalu tekan + Add Device
4. **Buat Token Baru**
 - a. **Pertama Connectkan dulu devicenya**
 - b. Klik **"+ Add Token"** maka langsung akan tercopy tokennya
5. Simpan Tokennya untuk di masukkan dalam monitor.py



MEMBUAT SCRIPT PYTHON (MONITOR.PY)

monitor.py

1. Masuk buat file monitor.py dalam local pc kita code programnya seperti di samping
2. Isi pada bagian Konfigurasi API Gemini. API yang telah kalian buat tadi.
3. Lalu pada def send_whatsapp(message). Masukkan token dari fonnte yang telah kita ambil tadi, dan masukkan nomor device yang telah kita hubungkan tadi.

```
import google.generativeai as genai
import datetime
import requests
import subprocess

# Konfigurasi API Gemini
genai.configure(api_key="API_KEY Gemini")

model = genai.GenerativeModel(model_name="gemini-1.5-flash")

def get_ssh_attempts():
    result = subprocess.check_output("grep 'Failed password' /var/log/auth.log | tail -n 10", shell=True)
    return result.decode()

def get_gemini_analysis(log_text):
    try:
        response = model.generate_content(f"Ada percobaan login brute force:\n{log_text}\nApa yang sebaiknya saya lakukan? responnya jangan terlalu panjang")
        return response.text
    except Exception as e:
        return f"⚠️ Gagal mendapatkan analisis dari Gemini: {e}"

def send_whatsapp(message):
    token = "Token_Fonnte"
    payload = {
        "target": "nomor hp yang akan digunakan",
        "message": message,
    }
    headers = {"Authorization": token}
    r = requests.post("https://api.fonnte.com/send", data=payload, headers=headers)
    return r.status_code

# Eksekusi semua
log = get_ssh_attempts()
ai_response = get_gemini_analysis(log)
full_message = f"[{datetime.datetime.now()}] ⚠️ Percobaan Login Detected!\n\n{log}\n\n🗨️ Gemini says:\n{ai_response}"
send_whatsapp(full_message)
```

UPLOADING MONITOR.PY TO GITHUB

UPLOAD

1. Kita membuat repo baru misalnya :
Monitoring
2. Lalu Kita upload file `monitor.py` ke dalam
github

CONFIGURING JENKINS FOR AUTOMATION

CONFIGURING JENKIN

1. Jalankan Jenkins dengan `sudo java -jar jenkins.war --httpPort=9090 &`
2. Login ke dalam Jenkinsnya
3. Klik new Item, lalu enter namanya. Sebebas kalian mau beri nama apa, jangan lupa select free style project
4. Lalu pada bagian Source Code Management :
 1. Isi ssh-key dari repository yang kita buat tadi
 2. Credentialnya juga
 3. Lalu Ganti type branch dari **master** menjadi **main**

Dashboard > Monitoring > Configuration

Configure

- General
- Source Code Management**
- Triggers
- Environment
- Build Steps
- Post-build Actions

Repositories

Repository URL ?
git@github.com:quiland15/Monitoring.git

Credentials ?
quiland (github)

+ Add

Advanced ▾

Add Repository

Branches to build

Branch Specifier (blank for 'any') ?
*/main

Add Branch

Save Apply

CONFIGURING JENKIN

1. Lalu pada bagian triggers tekan build periodically

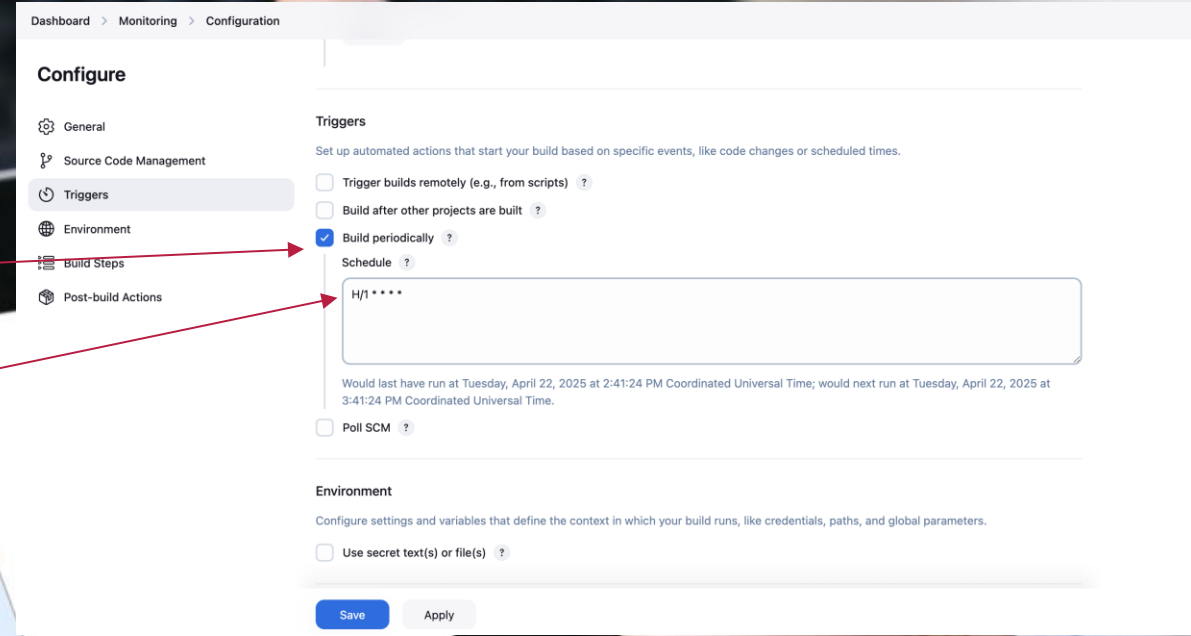
1. buat schedule untuk untuk berjalan setiap 1 hari sekali H/24 * * * *

2. Lalu kita buat build stepnya klik add build step lalu pilih execute shell

3. Masukkan command ini:

- `cd /home/quiland` (ini tergantung kalian ingin simpan reponya Dimana)
- `rm -rf Monitoring` (untuk menghapus repo yang lama)
- `git clone https://github.com/quiland15/Monitoring.git` (mengclone reponya)
- `cd Monitoring` (Masuk dalam reponya)
- `python3 monitor.py` (Menjalankan monitor.py)

4. Lalu kita save dan mencobanya



Dashboard > Monitoring > Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☒ Build periodically ?

Schedule ?

H/1 * * * *

Would last have run at Tuesday, April 22, 2025 at 2:41:24 PM Coordinated Universal Time; would next run at Tuesday, April 22, 2025 at 3:41:24 PM Coordinated Universal Time.

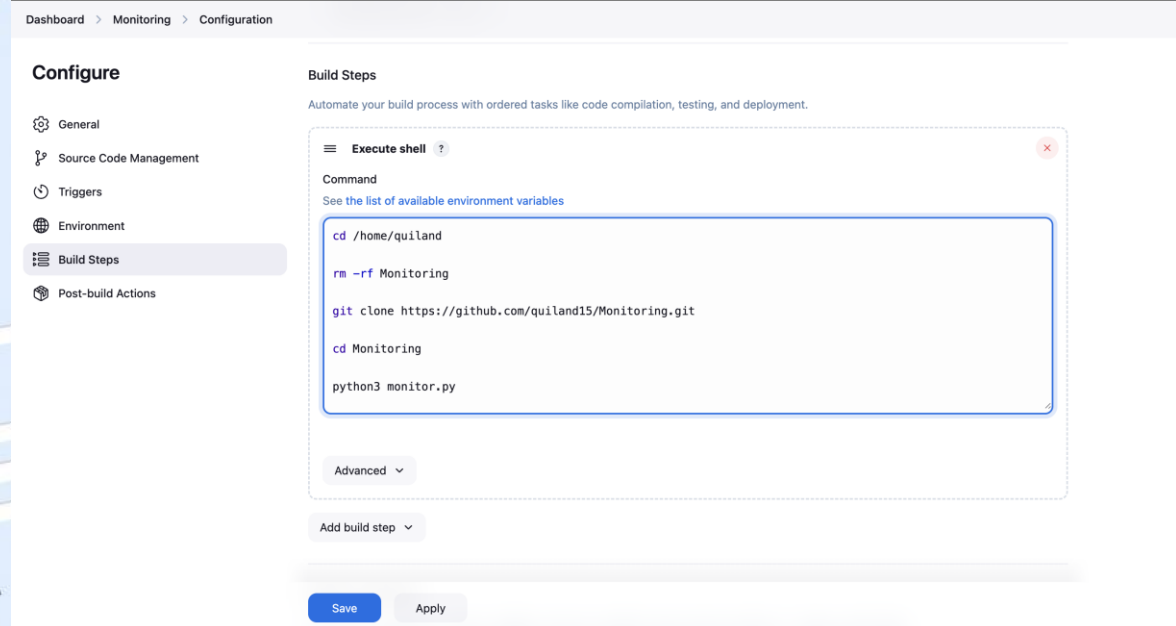
☐ Poll SCM ?

Environment

Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.

☐ Use secret text(s) or file(s) ?

Save Apply



Dashboard > Monitoring > Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute shell ?

Command

See the list of available environment variables

```
cd /home/quiland
rm -rf Monitoring
git clone https://github.com/quiland15/Monitoring.git
cd Monitoring
python3 monitor.py
```

Advanced ▾

Add build step ▾

Save Apply

A rectangular sign with a white background and a thin black border is mounted on a dark gray wall. The sign features the words "THANK YOU" in a large, bold, dark gray sans-serif font, centered horizontally. Below the sign, a wooden floor with light brown planks is visible.

THANK YOU